



MEMORIA TÉCNICA

Práctica Avanzada: Saneamiento y Restauración
de Base de Datos

Base de Datos: gha_analytics

Gestión Hospitalaria Avanzada

Abril 2026

Tabla de Contenidos

1. Resumen Ejecutivo

2. Normalización de Identidad (Pacientes)

2.1 Eliminación de Duplicados

2.2 Limpieza y Validación de NIF

2.3 Restricciones de Integridad

3. Consistencia de Colegiados (Médicos)

3.1 Estandarización de Formatos

3.2 Validación mediante CHECK

4. Integridad Referencial

4.1 Corrección de Especialidades

4.2 Implementación de Foreign Keys

5. Normalización de Tablas

6. Columnas Calculadas y Blindaje

7. Ingesta de Datos Externos

8. Limpieza Adicional de Datos

9. Optimización de Rendimiento

10. Conclusiones

1. Resumen Ejecutivo

El presente documento detalla el proceso completo de saneamiento y restauración de la base de datos **gha_analytics** del sistema de gestión hospitalaria. Se han implementado mejoras estructurales y de datos que garantizan la integridad, consistencia y rendimiento del sistema.

Alcance del Proyecto

El saneamiento abarcó cuatro tablas principales del sistema: `pacientes`, `medicos`, `visitas` y `especialidades`, además de la creación de la nueva tabla `seguros_pacientes` para normalizar la información de seguros.

Métricas del Proyecto

8

Bloques de cambios

4

Tablas modificadas

1

Tabla nueva creada

7

Índices creados

Objetivos Alcanzados

- Eliminación de registros duplicados en la tabla de pacientes
- Validación y estandarización de documentos de identidad (NIF)
- Normalización de números de colegiado médico
- Implementación de restricciones de integridad referencial
- Separación de datos de seguros a tabla independiente

1. Resumen Ejecutivo

- Cálculo automático de copagos estimados
- Limpieza de datos de contacto (emails y teléfonos)
- Optimización del rendimiento mediante índices estratégicos

2. Normalización de Identidad (Pacientes)

La tabla `pacientes` presentaba problemas de duplicidad y formatos inconsistentes en el campo NIF. Se implementaron las siguientes correcciones:

2.1 Eliminación de Duplicados

Tabla 1 Resumen de cambio - Eliminación de duplicados

Aspecto	Detalle
Cambio realizado	Eliminación de registros duplicados basados en NIF, manteniendo el ID más bajo
Justificación	Los registros duplicados generan inconsistencias en las consultas y reportes. Mantener el ID más bajo asegura la referencia histórica más antigua.
Método	Subconsulta con MIN(id) agrupado por NIF
Transacción	Sí - permite rollback en caso de error

```
DELETE FROM pacientes
WHERE id NOT IN (
    SELECT id_min FROM (
        SELECT MIN(id) AS id_min
        FROM pacientes
        GROUP BY nif
    ) AS tmp
);
```

2.2 Limpieza y Validación de NIF

Tabla 2 Resumen de cambio - Limpieza de NIF

Aspecto	Detalle
Cambio realizado	Eliminación de espacios y guiones del campo NIF
Justificación	Los espacios y caracteres especiales impiden la validación correcta y la creación de índices únicos.
Funciones utilizadas	TRIM(), REPLACE()

```
UPDATE pacientes SET nif = TRIM(nif);
UPDATE pacientes SET nif = REPLACE(nif, '-', '');
```

Tabla 3 Resumen de cambio - Validación de formato NIF

Aspecto	Detalle
Cambio realizado	Eliminación de registros con NIF que no cumplen el formato español
Justificación	El formato válido es 8 dígitos seguidos de una letra. Los registros con formato incorrecto son datos corruptos que deben eliminarse.
Patrón Regex	^[0-9]{8}[A-Za-z]\$

```
DELETE FROM pacientes
WHERE nif NOT REGEXP '^[0-9]{8}[A-Za-z]$';
```

2.3 Restricciones de Integridad

Tabla 4 Resumen de cambio - Restricciones NIF

Aspecto	Detalle
Cambio realizado	Conversión de columna nif a UNIQUE y NOT NULL
Justificación	Garantiza que cada paciente tenga un NIF único y obligatorio, evitando duplicados futuros y asegurando la identidad.
Restricciones	NOT NULL, UNIQUE (uq_nif)

```
ALTER TABLE pacientes
MODIFY nif VARCHAR(9) NOT NULL,
ADD CONSTRAINT uq_nif UNIQUE (nif);
```

3. Consistencia de Colegiados (Médicos)

La tabla `medicos` requería estandarización en los números de colegiado para garantizar la trazabilidad profesional.

3.1 Estandarización de Formatos

Tabla 5 Resumen de cambio - Estandarización de colegiados

Aspecto	Detalle
Cambio realizado	Estandarización de todos los números de colegiado al formato COL-XX-YYYY
Justificación	El formato COL-XX-YYYY (provincia-número) es el estándar oficial de los colegios médicos españoles. Permite identificar la provincia de colegiación.
Transformaciones	Reemplazo de '/' por '-', corrección de valores inválidos como 'INV-999'
Formato final	COL-[0-9]{2}-[0-9]{4}

```
UPDATE medicos SET num_colegiado = REPLACE(num_colegiado, '/', '-');
UPDATE medicos SET num_colegiado = 'COL-99-0999' WHERE num_colegiado = 'INV-999';
UPDATE medicos
SET num_colegiado = CONCAT('COL-', LEFT(REPLACE(num_colegiado, 'COL', ''), 2), '-',
RIGHT(num_colegiado, 4))
WHERE num_colegiado NOT REGEXP '^COL-[0-9]{2}-[0-9]{4}$';

DELETE FROM medicos
WHERE num_colegiado NOT REGEXP '^COL-[0-9]{2}-[0-9]{4}$';
```


3.2 Validación mediante CHECK

Tabla 6 Resumen de cambio - Restricción CHECK

Aspecto	Detalle
Cambio realizado	Adición de restricción CHECK para validar el formato de colegiado
Justificación	Previene la inserción de números de colegiado con formato incorrecto en el futuro, manteniendo la consistencia de datos.
Restricción	chk_num_colegiado

```
ALTER TABLE medicos
ADD CONSTRAINT chk_num_colegiado
CHECK (num_colegiado REGEXP '^COL-[0-9]{2}-[0-9]{4}$');
```

4. Integridad Referencial

La integridad referencial es fundamental para mantener la consistencia entre tablas relacionadas y evitar registros huérfanos.

4.1 Corrección de Especialidades

Tabla 7 Resumen de cambio - Corrección de especialidades

Aspecto	Detalle
Cambio realizado	Asignación de 'Medicina General' a médicos con especialidades inexistentes
Justificación	Los médicos deben tener una especialidad válida. Asignar 'Medicina General' como valor por defecto asegura que todos los registros sean válidos antes de crear la FK.
Método	Subconsulta para obtener el ID de 'Medicina General'

```
UPDATE medicos
SET especialidad_id = (SELECT id FROM especialidades WHERE nombre = 'Medicina
General')
WHERE especialidad_id NOT IN (SELECT id FROM especialidades);
```

4.2 Implementación de Foreign Keys

Tabla 8 Resumen de cambio - Foreign Keys implementadas

Tabla	Foreign Key	Referencia	Justificación
medicos	fk_medicos_especialidades	especialidades(id)	Garantiza que cada médico tenga una especialidad válida registrada
visitas	fk_visitas_pacientes	pacientes(id)	Asegura que cada visita esté asociada a un paciente existente
visitas	fk_visitas_medicos	medicos(id)	Asegura que cada visita esté asociada a un médico existente

Nota importante

Antes de crear las foreign keys, se eliminaron las visitas huérfanas (aquellas cuyo paciente_id o medico_id no existían en sus tablas respectivas) para evitar el Error 1452 de MySQL.

```
-- Limpieza previa de visitas huérfanas
DELETE FROM visitas WHERE paciente_id NOT IN (SELECT id FROM pacientes);
DELETE FROM visitas WHERE medico_id NOT IN (SELECT id FROM medicos);

-- Creación de Foreign Keys
ALTER TABLE medicos
ADD CONSTRAINT fk_medicos_especialidades
FOREIGN KEY (especialidad_id) REFERENCES especialidades(id)
ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE visitas
ADD CONSTRAINT fk_visitas_pacientes
FOREIGN KEY (paciente_id) REFERENCES pacientes(id)
ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE visitas
ADD CONSTRAINT fk_visitas_medicos
FOREIGN KEY (medico_id) REFERENCES medicos(id)
ON DELETE RESTRICT ON UPDATE CASCADE;
```

5. Normalización y División de Tablas

Se aplicó la Tercera Forma Normal (3FN) separando los datos de seguros de la tabla pacientes a una tabla independiente.

Tabla 9 Resumen de cambio - Nueva tabla seguros_pacientes

Aspecto	Detalle
Cambio realizado	Creación de tabla seguros_pacientes independiente
Justificación	La información de seguros es independiente de los datos del paciente. Un paciente puede tener múltiples pólizas o cambiar de aseguradora sin modificar sus datos personales.
Campos	id, paciente_id, num_poliza, estado_poliza (default 'ACTIVA')
Integridad	Foreign Key a pacientes(id) con ON DELETE RESTRICT

```
CREATE TABLE seguros_pacientes (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  paciente_id INT          NOT NULL,
  num_poliza  VARCHAR(50) NOT NULL,
  estado_poliza VARCHAR(20) DEFAULT 'ACTIVA',
  CONSTRAINT fk_seguros_paciente
    FOREIGN KEY (paciente_id) REFERENCES pacientes(id)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Migración de datos existentes
INSERT INTO seguros_pacientes (paciente_id, num_poliza)
SELECT id, num_poliza FROM pacientes WHERE num_poliza IS NOT NULL;

-- Eliminación de columna de la tabla original
ALTER TABLE pacientes DROP COLUMN num_poliza;
```

6. Columnas Calculadas y Blindaje

Se implementó el cálculo automático del copago y se establecieron restricciones de obligatoriedad para garantizar la integridad de los datos financieros.

6.1 Saneamiento del Importe

Tabla 10 Resumen de cambio - Limpieza de importes

Aspecto	Detalle
Cambio realizado	Limpieza de la columna importe_sucio y conversión a DECIMAL
Justificación	Los importes contenían símbolos de moneda, espacios y separadores decimales inconsistentes que impedían operaciones matemáticas.
Limpiezas aplicadas	Eliminación de €, \$, EUR, espacios, comas; reemplazo por punto decimal

```
UPDATE visitas SET importe_sucio = REPLACE(importe_sucio, '€', '');
UPDATE visitas SET importe_sucio = REPLACE(importe_sucio, '$', '');
UPDATE visitas SET importe_sucio = REPLACE(importe_sucio, 'EUR', '');
UPDATE visitas SET importe_sucio = TRIM(importe_sucio);
UPDATE visitas SET importe_sucio = REPLACE(importe_sucio, ' ', '');
UPDATE visitas SET importe_sucio = REPLACE(importe_sucio, ',', '.');
UPDATE visitas SET importe_sucio = '0.00' WHERE importe_sucio REGEXP '[a-zA-Z]+';

-- Conversión a tipo DECIMAL
ALTER TABLE visitas MODIFY COLUMN importe_sucio DECIMAL(10,2);
ALTER TABLE visitas RENAME COLUMN importe_sucio TO importe;
```

6.2 Cálculo del Copago Estimado

Tabla 11 Resumen de cambio - Columna copago_estimado

Aspecto	Detalle
Cambio realizado	Creación de columna copago_estimado calculada como 20% del importe
Justificación	El copago del 20% es una regla de negocio estándar en el sistema de salud. Calcularlo automáticamente evita errores manuales.
Fórmula	$\text{copago} = \text{ROUND}(\text{importe} * 0.20, 2)$
Blindaje	NOT NULL - todos los registros deben tener copago calculado

```
ALTER TABLE visitas ADD COLUMN copago_estimado DECIMAL(10,2);

UPDATE visitas SET copago_estimado = ROUND(importe * 0.20, 2);

-- Blindaje NOT NULL
ALTER TABLE visitas MODIFY COLUMN copago_estimado DECIMAL(10,2) NOT NULL;
```

7. Ingesta de Datos Externos

Se implementó un proceso ETL (Extract, Transform, Load) para importar datos desde la tabla de staging `raw_import_visitas` .

Tabla 12 Resumen de cambio - Proceso ETL

Aspecto	Detalle
Cambio realizado	Procesamiento e importación de datos desde raw_import_visitas
Justificación	Los datos externos requieren limpieza, validación y transformación antes de integrarse en las tablas principales del sistema.
Prevención duplicados	Verificación de NIF existente antes de insertar nuevos pacientes
Formatos de fecha	Soporte múltiple: dd/mm/yyyy, yyyy-mm-dd, dd-mm-yyyy, yyyy.mm.dd


```

-- Insertar pacientes nuevos (no existentes por NIF)
INSERT INTO pacientes (nif, nombre_completo)
SELECT DISTINCT
    UPPER(TRIM(SUBSTRING_INDEX(raw_data, '|', 1))),
    TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(raw_data, '|', 2), '|', -1))
FROM raw_import_visitas
WHERE UPPER(TRIM(SUBSTRING_INDEX(raw_data, '|', 1)))
    NOT IN (SELECT nif FROM pacientes);

-- Insertar visitas con conversión de formatos
INSERT INTO visitas (paciente_id, medico_id, fecha_visita, importe,
copago_estimado)
SELECT
    p.id,
    1,
    CASE
        WHEN fecha_raw LIKE '%/%/____' THEN STR_TO_DATE(fecha_raw, '%d/%m/%Y')
        WHEN fecha_raw LIKE '____-__-__' THEN STR_TO_DATE(fecha_raw, '%Y-%m-%d')
        WHEN fecha_raw LIKE '__-__-____' THEN STR_TO_DATE(fecha_raw, '%d-%m-%Y')
        WHEN fecha_raw LIKE '____.%.%' THEN STR_TO_DATE(fecha_raw, '%Y.%.%.d')
    END,
    CAST(importe_limpio AS DECIMAL(10,2)),
    ROUND(CAST(importe_limpio AS DECIMAL(10,2)) * 0.20, 2)
FROM raw_import_visitas r
JOIN pacientes p ON p.nif = UPPER(TRIM(SUBSTRING_INDEX(r.raw_data, '|', 1)));

```

8. Limpieza Adicional de Datos

Se realizaron limpiezas adicionales en campos de contacto y fechas para garantizar la calidad de los datos.

8.1 Corrección de Emails

Tabla 13 Resumen de cambio - Corrección de emails

Problema	Solución	Justificación
gmail,con → gmail.com	REPLACE(email, 'gmail,con', 'gmail.com')	Error tipográfico común en dominios
.con → .com	REPLACE(email, '.con', '.com')	Extensión de dominio incorrecta
Emails con doble @	SET email = NULL	Formato inválido, no recuperable

8.2 Normalización de Teléfonos

Tabla 14 Resumen de cambio - Normalización de teléfonos

Transformación	Resultado
Eliminar espacios	"600 123 456" → "600123456"
Eliminar guiones	"600-123-456" → "600123456"
Eliminar prefijo +34	" +34600123456" → "600123456"
Convertir 0034	"0034600123456" → "600123456"

8.3 Conversión de Tipos de Datos

Tabla 15 Resumen de cambio - Conversiones de tipo

Columna	Tipo Original	Tipo Final	Justificación
f_nacimiento	VARCHAR	DATE	Permite cálculos de edad y validaciones
fecha_visita	VARCHAR	DATETIME	Almacena fecha y hora de la visita
descuento_aplicado	VARCHAR	DECIMAL(10,2)	Permite operaciones matemáticas

9. Optimización de Rendimiento

Se crearon índices estratégicos para mejorar el rendimiento de las consultas más frecuentes, aplicando el principio de SARGability (Search ARGument ABLE).

Tabla 16 Índices creados

Nombre del Índice	Tabla	Columna(s)	Justificación
idx_visitas_fecha	visitas	fecha_visita	Consultas por rango de fechas (informes mensuales)
idx_visitas_paciente	visitas	paciente_id	Historial de visitas por paciente
idx_visitas_medico	visitas	medico_id	Agenda y estadísticas por médico
idx_medicos_especialidad	medicos	especialidad_id	Búsqueda de médicos por especialidad

Consulta SARGable

El siguiente ejemplo muestra una consulta que utiliza el índice de fecha de manera eficiente, evitando funciones en el WHERE que invalidarían el uso del índice:

```
SELECT * FROM visitas
WHERE fecha_visita >= '2026-03-01' AND fecha_visita < '2026-04-01';
```

10. Conclusiones

El proceso de saneamiento y restauración de la base de datos **gha_analytics** ha sido completado exitosamente, logrando los siguientes resultados:

Logros Principales

- **Integridad de datos:** Todos los registros cumplen con las restricciones de formato y relaciones definidas.
- **Estandarización:** Los campos críticos (NIF, colegiados, teléfonos, emails) siguen formatos consistentes.
- **Normalización:** La separación de seguros a tabla independiente reduce la redundancia y mejora la flexibilidad.
- **Automatización:** El cálculo automático del copago elimina errores manuales.
- **Rendimiento:** Los índices creados optimizan las consultas más frecuentes.

Medidas de Seguridad Implementadas

- Uso de transacciones (START TRANSACTION / COMMIT) para garantizar la atomicidad de las operaciones
- Desactivación temporal de SQL_SAFE_UPDATES para operaciones controladas
- Restricciones CHECK para validación de formatos
- Foreign Keys con ON DELETE RESTRICT para prevenir eliminaciones accidentales
- Campos NOT NULL para datos obligatorios

Recomendaciones Futuras

- Implementar validación del dígito de control del NIF (letra)
- Crear triggers para auditoría de cambios en registros sensibles
- Establecer políticas de backup automatizado
- Implementar monitoreo de rendimiento de consultas frecuentes

Documentación Técnica

Esta memoria técnica debe mantenerse actualizada ante cualquier modificación futura en la estructura de la base de datos. Se recomienda versionar los cambios SQL siguiendo el mismo esquema de bloques numerados utilizado en este documento.